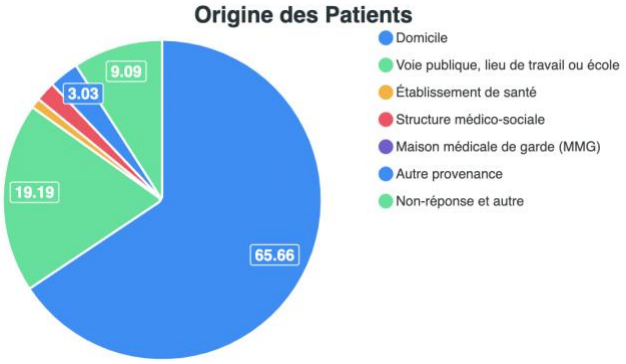
TP Informatique – Vue

Graphiques et cartographie

Étape 0 : Objectifs

Graphiques : Le premier objectif est de créer une application permettant de réaliser des graphiques pour visualiser un jeu de données.



Pour cela il faut choisir une librairie JS ; parmi les plus utilisées, nous allons retenir ApexChart https://apexcharts.com/ .
Cette librairie propose un module « vue-apexcharts » pour simplifier son usage dans Vue.
L'installation dans une application Vue se fera donc à l'aide des 2 commandes :

```
npm install apexcharts
npm install vue3-apexcharts
```

Dans « Stackblitz » il faut utiliser le terminal ;
(« ctrl + C » pour interrompre « vite »
puis « npm run dev » pour relancer)

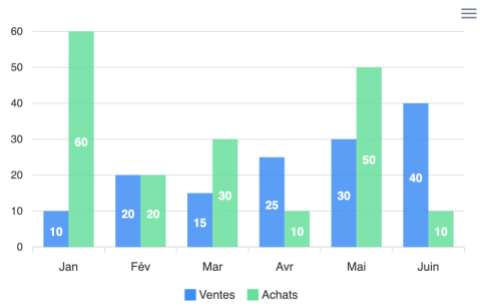
Exemple de code pour obtenir ce graphique :

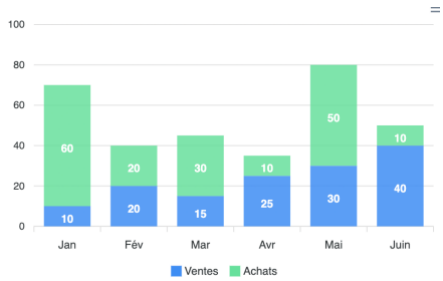
```
<script setup>
 import apexchart from "vue3-apexcharts";
 // -- les series de données à afficher
```

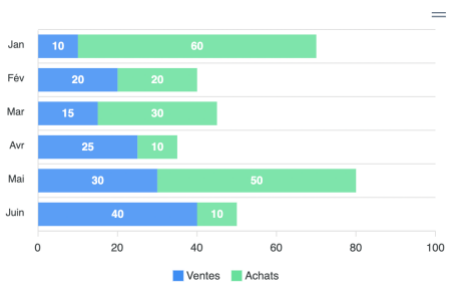
```
 const series = [
  {
   name: "Ventes",
   data: [ { x: "Jan", y: 10 }, { x: "Fév", y: 20 },{ x: "Mar", y: 15 },
    { x: "Avr", y: 25 }, { x: "Mai", y: 30 }, { x: "Juin", y: 40 }
   ]
  },
  {
   name: "Achats",
   data: [ { x: "Jan", y: 60 }, { x: "Fév", y: 20 },{ x: "Mar", y: 30 },
    { x: "Avr", y: 10 }, { x: "Mai", y: 50 }, { x: "Juin", y: 10 }
   ]
  }
 ];
 // -- les options du graphique
 const options = {
  chart: {
   type : "line", // type de graphique
  },
  stroke: {
   curve: 'smooth', // type de ligne
  },
  markers: {
   size: 5, // taille des marqueurs à chaque point
  },
 }
</script>

<template>
 <div>
  <apexchart width="500" :options="options" :series="series"></apexchart>
 </div>
</template>
```

Il est possible de le transformer en :







Pour le premier, il faut changer les options par :

```
const options = {
  chart: {
    type : "bar",
  },
}
```

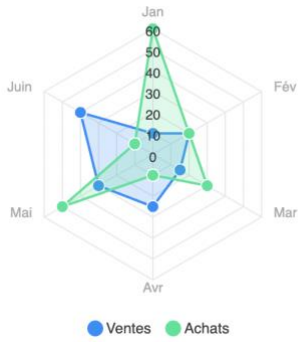
Pour le second :

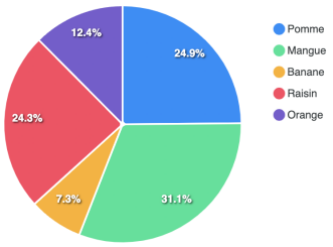
```
const options = {
  chart: {
    type : "bar", // type de graphique
    stacked : true, // empilement des barres
  },
```

Pour le troisième :

```
const options = {
  chart: {
    type : "bar", // type de graphique
    stacked : true, // empilement des barres
  },
  plotOptions: {
    bar: {   horizontal: true }
  }
}
```

Il est aussi possible de faire un graphique mixte, en radar ou encore en camembert :





Pour le graphique mixte, sur le code précédent, Il faut préciser l'orientation pour chaque série de données :

```
const series = [
  {
    name: "Ventes",
    type: 'column',
    data: [ … ]
  },
  {
    name: "Achats",
    type: 'line',
    data: [ … ]
  }
];
```

Avec une option pour afficher les valeurs des données sur le graphique :

```
const options = {
   dataLabels: {
     enabled: true,
   },
 }
```

Pour le radar, il faut juste préciser : type : "radar" (et enlever type : 'column' et type : 'line')

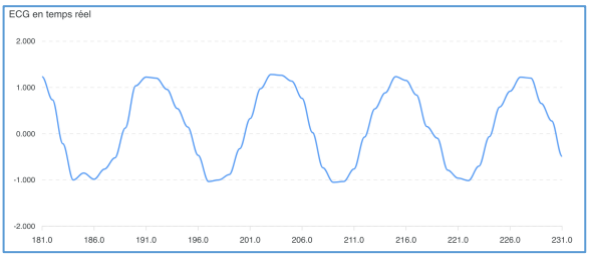
Pour le camembert, voici un exemple avec des données différentes :

```
<script setup>
 import apexchart from "vue3-apexcharts";
 // -- les données à afficher
 const seriesData = [
    { label: "Pomme", value: 44 },
    { label: "Mangue", value: 55 },
    { label: "Banane", value: 13 },
    { label: "Raisin", value: 43 },
    { label: "Orange", value: 22 },
  ];
 const series = seriesData.map(item => item.value) // Extraction des valeurs
 const labels = seriesData.map(item => item.label) // Extraction des labels

 // -- les options du graphique
 const options = {
  chart: {
    type : "pie", // type de graphique
  },
  labels: labels, // Extraction des labels
 }
```

```
</script>
```

Affichage d'un ECG en temps réel (simulé) :




```
<template>
  <div>
    <apexchart height="350" width="80%" :options="chartOptions" :series="series" ></apexchart>
  </div>
</template>

<script setup>
 import { ref, onMounted } from "vue";
 import apexchart from "vue3-apexcharts";

 const chartOptions = {
  chart: { type: "line", toolbar: { show: false },},
  stroke: { curve: "smooth", width: 2,},
  xaxis: { type: "numeric", tickAmount: 10,}, // Nombre de divisions sur l'axe X
  yaxis: { min: -2,max: 2,}, // Amplitude simulée d'un ECG
  title: { text: "ECG en temps réel" },
  grid: { borderColor: "#e7e7e7", strokeDashArray: 5,},
 };

 // Données du graphique (initialisées vides)
 const series = ref([ { name: "ECG", data: []} ]);
 // Fonction pour générer un signal ECG simulé
 const generateECGSignal = () => {
  return (Math.sin(Date.now() + Math.random() * 0.5) * 1.2 + (Math.random() * 0.2)).toFixed(3)
 };
 // Mise à jour dynamique du tracé ECG
 onMounted(() => {
  let x = 0;
  setInterval(() => {
   const newPoint = { x: x++, y: generateECGSignal() }; // un nouveau point
   if (series.value[0].data.length > 50) {
    series.value[0].data.shift(); // Supprime le point le plus ancien
```

```
    }
    series.value[0].data.push(newPoint); // ajoute le nouveau point dans les datas
  }, 100); // Mise à jour toutes les 100 ms
});
</script>

<style scoped></style>
```

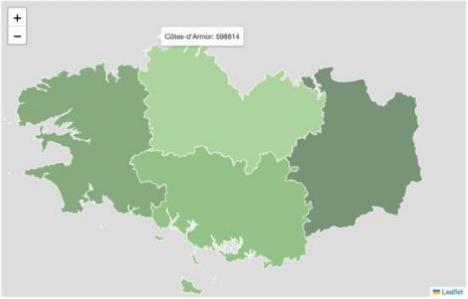
→ Les possibilités sont très nombreuses, à vous d'explorer la documentation !!

Cartographie : Le deuxième objectif est de pouvoir visualiser des données sur une carte.

Là aussi il faut choisir une librairie JS ;
Nous utiliserons **LeafLet** https://leafletjs.com/
Pour son intégration dans vue, nous n'utiliserons pas de module spécifique.
L'installation se fera par une seule commande :

```
npm install leaflet
```

Leaflet n'est qu'une librairie JS ; elle a besoin de fonds de cartes comme ceux proposés par **OpenStreetMap**.
https://www.openstreetmap.org/
OSM est un fournisseur de fonds de carte comme Google Maps, Bing Maps ou encore IGN.OSM se distingue cependant de ces différents organismes par sa démarche. En effet, toutes les cartes sont construites de manière collaborative et sont placées sous licence libre.



Exemple de composant Vue pour visualiser des cabinets médicaux de Castres (cf. illustration ci-dessus)

```
<template>
  <div>
    <!-- il faut prévoir une div pour afficher la carte -->
    <!-- et il faut qu'elle est une dimension non null ==> style -->
    <div id="mapcarto" class="map-container">
    </div>
  </div>
</template>

<script setup>
  import { onMounted } from "vue";
  // --- importer Leaflet et le css associé
  import L from "leaflet";
  import "leaflet/dist/leaflet.css";
  // --  datas à afficher sur la carte
  const cabinetsMedical = [
    { "nom" : "Maison de Santé Carré Gambetta", "lat" : 43.60411, "lng" : 2.23862 },
    { "nom" : "Cabinet Médical Gaillac et Luguet", "lat" : 43.60883, "lng" : 2.23078 },
    { "nom" : "Cabinet Médical de Lameilhé", "lat" : 43.59259, "lng" : 2.25492 },
    { "nom" : "Cabinet Médical du Saillenc", "lat" :  43.60928, "lng" : 2.25946 }
  ]
  // -- Facultatif : définir une icône personnalisée pour les centres médicaux
  const myIcon = L.icon({
     iconUrl: 'https://cdn-icons-png.flaticon.com/512/387/387561.png',
     iconSize: [48, 48],
  });

  // ------------  Fonction pour afficher la carte
  function afficheCarto(){
     // Initialisation de la carte avec leaftlet
     //   - la placer dans la div d'id 'mapcarto'
     //   - la centrer sur Castres avec un zoom de 14
     let mapcarto = L.map('mapcarto').setView([43.60548, 2.24167], 14); // France

     // Ajouter un fond de carte (OpenStreetMap)
     L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
       attribution: '© OpenStreetMap contributors'
     }).addTo(mapcarto);

     // Ajouter les markers :  1 pour chaque centre médical
```

```
      cabinetsMedical.forEach((cabinet) => {
        L.marker([cabinet.lat, cabinet.lng], {icon: myIcon}).addTo(mapcarto)
        .bindPopup(cabinet.nom); // Ajouter une popup au marker
      });

      // Ajouter un cercle sur la carte
      L.circle([43.60548, 2.24167], {
        color: 'blue', fillOpacity: 0.1, radius: 2000
      }).addTo(mapcarto);
    }
  // -- afficher la carte à la création du composant
  onMounted(afficheCarto);
</script>

<style scoped>
  /* il faut que la div ait une dimension non null */
   .map-container { width: 80%; height: 700px; margin: 20px;}
</style>
```

Leaflet peut afficher aussi une couche de données au format GeoJSON[1]. Par exemple sur l'illustration précédente, on affiche le contour administratif des départements de la Bretagne. On peut ainsi colorer chaque département en fonction de la valeur d'un indicateur.

Un document au format GeoJSON a pour structure *(attention ce n'est qu'un extrait)* :

```
{
  "type": "FeatureCollection",
  "features": [     // -- chaque feature (elt du tableau features) correspond au contour du dept
    {
      "type": "Feature",
      "geometry": {
        "type": "Polygon",  // -- le contour est défini comme un polygone
        "coordinates": [
          [ [ 4.78021, 46.17668], [ 4.78024, 46.18905], … ]
        ]
      },
      "properties": {  // -- on peut ici définir les propriétés de chaque dept.
        "code": "81",
        "nom": "Tarn"
      }
    }
  ]
}
```

Sur ce site, on peut trouver de nombreux fonds de cartes de la France au format GeoJSON avec plusieurs découpages administratifs : https://github.com/gregoiredavid/france-geojson

Pour la bretagne, vous pouvez télécharger le fichier ici :
https://pecatte.fr/tpweb/cartographie/departementBretagne.json

➔ Récupérer le fichier et le stocker dans le dossier « src/assets/ » en conservant son nom.

[1] *GeoJSON est un format permettant de coder une variété de structures de données géographiques […]. Un objet GeoJSON peut représenter une région de l'espace (une géométrie), une entité délimitée spatialement (une entité) ou une liste d'entités (une FeatureCollection). GeoJSON prend en charge les types de géométrie suivants : Point, LineString, Polygon, MultiPoint, MultiLineString, MultiPolygon et GeometryCollection. Les entités dans GeoJSON contiennent un objet Geometry et des propriétés supplémentaires, et une FeatureCollection contient une liste d'entités.*

Exemple de composant Vue pour visualiser le nombre d'habitants de chaque département de la Bretagne (cf. illustration ci-dessus) :

```
<template>
  <div>
    <div id="mapBretagne" class="map-container">
    </div>
  </div>
</template>

<script setup>
  import { onMounted } from "vue";
  // --- importer Leaflet et le css associé
  import L from "leaflet";
  import "leaflet/dist/leaflet.css";

  // -- Fichier GeoJSON des départements de Bretagne
  import geojsonBretagne from "../assets/departementBretagne.json";

  // -- nb habitants des depts de Bretagne
  const nbHabitantsBretagne = [
     {code: "22", nbHabitants: 598814}, {code: "29", nbHabitants: 927912},
     {code: "35", nbHabitants: 1109232}, {code: "56", nbHabitants: 776103}
  ]
  // -- Fonction pour colorer les départements en fonction du nombre d'habitants '
  const getColor = (count) => {
    return count > 1000000 ? "#0c4a0b" : count > 900000 ? "#287919" :
           count > 700000 ? "#43a926" : "#7bcc5d";
  };
  // -- affiche la carte de la Bretagne
  function afficheCarto(){
    // Initialisation de la carte avec leaftlet
    let map = L.map('mapBretagne').setView([48.19107, -2.773595],8); // Bretagne
    // Afficher la couche GeoJSON
    L.geoJSON(geojsonBretagne, {
       style:
         // -- fonction qui permet de définir un style différent pour chaque département
        (feature) => {
         const deptCode = feature.properties.code;
         // -- nb d'habitants par departement
              //          qu'il faut chercher dans le tableau nbHabitantsBretagne
         const nbHabitants = nbHabitantsBretagne.find((v)=>v.code==deptCode).nbHabitants || 0
```

```
        return {
          fillColor: getColor(nbHabitants), // -- couleur en fonction du nb habitants
          weight: 1, opacity: 1,color: "white",fillOpacity: 0.5,
        }
      },
      // -- fonction que s'exécute pour chaque feature (chaque département)
      //    pour ajouter un tooltip avec le nom du département et le nb d'habitants
      onEachFeature: (feature, layer) => {
        const deptCode = feature.properties.code;
        const nbHabitants = nbHabitantsBretagne.find((v)=>v.code==deptCode).nbHabitants || 0
        layer.bindTooltip(`${feature.properties.nom}: ${nbHabitants}`);
      }
    }).addTo(map);
  }
  onMounted(afficheCarto);
</script>

<style scoped>
  /* il faut que la div qui contient la map ait une dimension non null */
  .map-container { width: 80%; height: 500px; margin: 20px;}
</style>
```

Étape 1 : Tester les exemples dans une nouvelle application Vue ; penser à installer les modules complémentaires comme indiqué à l'étape 0.

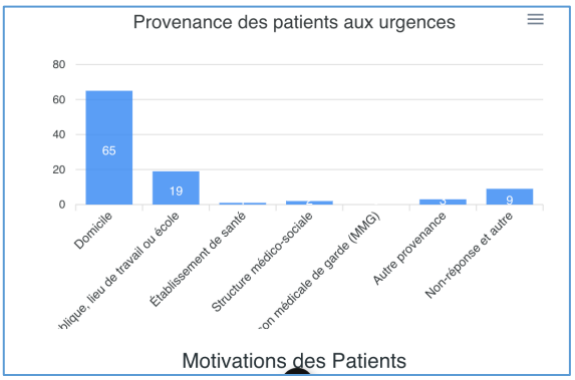
Étape 2 : Réalisation de graphiques statistiques

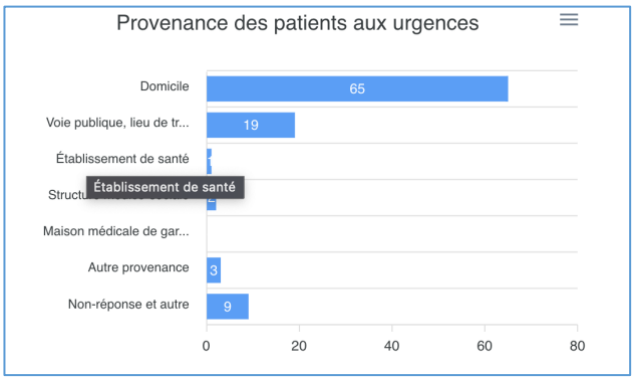
A) Provenance des patients aux urgences :

- Voici un jeu de données :

```
const seriesData = [
 { name: 'Provenance',
    data : [
    { x : 'Domicile', y: 65 },
    { x : 'Voie publique, lieu de travail ou école', y: 19 },
    { x : 'Établissement de santé', y: 1 },
    { x : 'Structure médico-sociale', y: 2 },
    { x : 'Maison médicale de garde (MMG)', y: 0 },
    { x : 'Autre provenance', y: 3 },
    { x : 'Non-réponse et autre', y: 9 },
 ]
}]
```

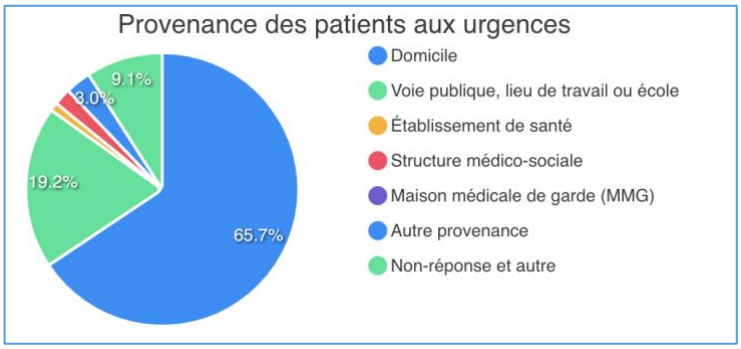
- Tester plusieurs types de graphique :





- Pour un graphique « camembert » il faut structurer les données autrement, en construisant un tableau qui contient les valeurs et un autre qui contient les étiquettes :

```
const series = seriesData[0].data.map(item => item.y) // Extraction des valeurs
const labels = seriesData[0].data.map(item => item.x) // Extraction des labels
```



B) Sortie des patients aux urgences :

Proposer un graphique pour ce jeu de données :

```
const seriesData = [
 { name: 'Sorties',
    data : [
    { x: 'Retour à domicile', y: 75.7 },
    { x: 'Retour en HAD', y: 0.6 },
    { x: 'Hospitalisation', y: 19.9 },
    { x: 'Retour en EHPAD', y: 0.9 },
    { x: 'Décès', y: 0.1 },
    { x: 'Sortie contre avis médical', y: 0.5 },
    { x: 'Vers une maison médicale de garde (MMG)', y: 0.2 },
    { x: 'Parti sans attendre', y: 2.0 }
 ]
}]
```

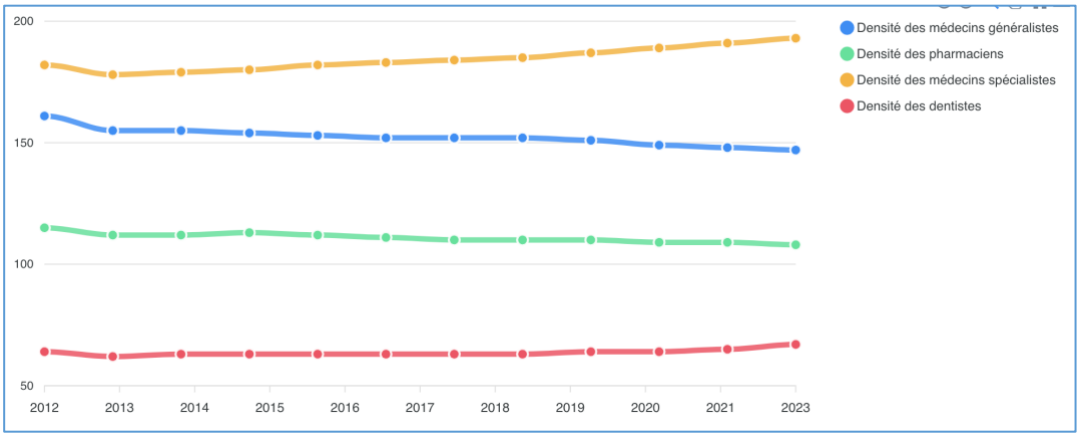
C) Motivation des patients pour venir aux urgences :

Proposer un graphique pour ce jeu de données :

```
const seriesData = [
  { name: 'Motivations',
     data : [
     { x: 'Accident', y: 31.9 },
     { x: 'Conseil médical', y: 26.0 },
     { x: 'État grave patient', y: 21.8 },
     { x: 'Angoisse patient', y: 9.8 },
     { x: 'Besoin régler rapidement', y: 27.3 },
     { x: 'Examen complémentaire', y: 23.1 },
     { x: 'Proximité urgences', y: 21.8 },
     { x: 'Consultation spécialiste', y: 11.9 },
     { x: 'Urgences plus rapides', y: 7.9 },
     { x: 'Médecin traitant absent', y: 5.9 },
     { x: 'Besoin examen sans RDV', y: 5.1 },
     { x: 'Amené par pompiers/SMUR', y: 11.3 }
  ]
 }]
```

D) Évolution de nombre de généralistes, spécialistes, pharmaciens, dentistes sur les 12 dernières années :

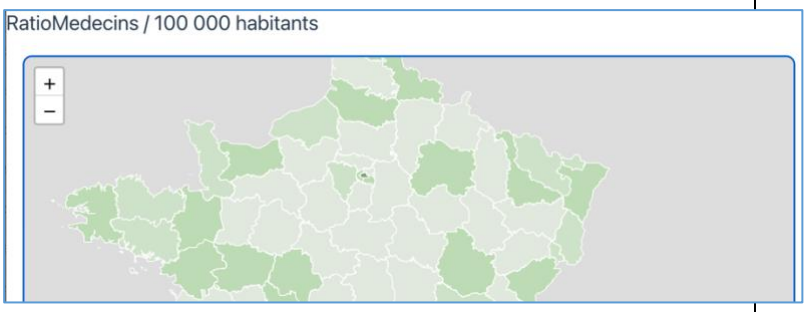
```
[
  {
    "name": "Densité des médecins généralistes",
    "data": [
      { "x": 2012, "y": 161 }, { "x": 2013, "y": 155 }, { "x": 2014, "y": 155 },
      { "x": 2015, "y": 154 }, { "x": 2016, "y": 153 }, { "x": 2017, "y": 152 },
      { "x": 2018, "y": 152 }, { "x": 2019, "y": 152 }, { "x": 2020, "y": 151 },
      { "x": 2021, "y": 149 }, { "x": 2022, "y": 148 }, { "x": 2023, "y": 147 }
    ]
  },
  {
    "name": "Densité des pharmaciens",
    "data": [
      { "x": 2012, "y": 115 }, { "x": 2013, "y": 112 }, { "x": 2014, "y": 112 },
      { "x": 2015, "y": 113 }, { "x": 2016, "y": 112 }, { "x": 2017, "y": 111 },
      { "x": 2018, "y": 110 }, { "x": 2019, "y": 110 }, { "x": 2020, "y": 110 },
      { "x": 2021, "y": 109 }, { "x": 2022, "y": 109 }, { "x": 2023, "y": 108 }
    ]
  },
  {
    "name": "Densité des médecins spécialistes",
    "data": [
      { "x": 2012, "y": 182 }, { "x": 2013, "y": 178 }, { "x": 2014, "y": 179 },
      { "x": 2015, "y": 180 }, { "x": 2016, "y": 182 }, { "x": 2017, "y": 183 },
      { "x": 2018, "y": 184 }, { "x": 2019, "y": 185 }, { "x": 2020, "y": 187 },
      { "x": 2021, "y": 189 }, { "x": 2022, "y": 191 }, { "x": 2023, "y": 193 }
    ]
  },
  {
    "name": "Densité des dentistes",
    "data": [
      { "x": 2012, "y": 64 }, { "x": 2013, "y": 62 }, { "x": 2014, "y": 63 },
      { "x": 2015, "y": 63 }, { "x": 2016, "y": 63 }, { "x": 2017, "y": 63 },
      { "x": 2018, "y": 63 }, { "x": 2019, "y": 63 }, { "x": 2020, "y": 64 },
      { "x": 2021, "y": 64 }, { "x": 2022, "y": 65 }, { "x": 2023, "y": 67 }
    ]
  }
]
```



E) Nombre de généralistes, spécialistes, pharmaciens, dentistes par département en 2023 :

Un extrait du fichier de données (le complet est disponible sur moodle) :


```
[
  {
    "code": "01",
    "nomdept": "Ain",
    "nbMedecins": "1162",
    "RatioMedecins": "174",
    "RatioGeneralistes": "99",
    "RatioSpecialistes": "75",
    "RatioDentistes": "53",
    "RatioPharmaciens": "78"
  },
...
```

- ✓ Réaliser une ou plusieurs cartes pour visualiser cette répartition géographique.
  Remarque : Le fond de carte de la France avec les départements est disponible sur moodle.

- ✓ Pour l'échelle des couleurs, il vous faut redéfinir la fonction getColor() (cf. exemple carto Bretagne ci-dessus) avec plus de couleurs et aussi adapter les échelles de valeur en fonction des données visualisées.
  Par exemple, le ratio des généralistes varie dans [86, 291] alors que pour les dentistes, il varie dans [33, 152].